

Computer Algebra for Cryptography: RSA and Polynomial Cryptosystem

Gust De Wit — r0948039

31 March 2026

1 Generalizing Pollard's $p - 1$ method

1.1 Pollard's $p + 1$ method

To test if a random element of the form $\alpha + \beta w$, with w a root, in the group $G = \frac{\mathbb{F}_2^\times}{\mathbb{F}_p^\times}$ is a neutral element, we must first explain what it means to be a neutral element in this context. Because the group G is a quotient group, the neutral element is the coset of elements in $\mathbb{F}_p^\times$, for which multiplying any element of G with an element of this coset will yield the same element in G . In other words, following from how cosets are defined, the neutral element is the set of all elements in $\mathbb{F}_{p^2}^\times$ that are equivalent to an element in $\mathbb{F}_p^\times$. This means that for it to be a neutral element, the coefficient β must be 0 mod p .

Even though we do not know p , we can still do computations in G by computing everything mod N instead of mod p . To then test if an element is a neutral element, we calculate $\gcd(\beta, N)$. If $\gcd(\beta, N) > 1$, we know, with certainty, that this divisor is a non-trivial factor of N , namely p or q . However, in the unlikely case that both $p + 1$ and $q + 1$ are B-smooth, it returns N . Otherwise, we know β is not 0 mod p , and the element is not the neutral element.

To know the expected number of polynomials $f(x) = x^2 + a$ we need to try in order to find an irreducible one, we consider that $f(x)$ is irreducible *iff* $-a$ is not a square. Since any element in $\mathbb{F}_p^\times$ can be written as g^k for some generator g and some integer $k \leq p - 1$, we know it is not a square for all odd k . This equates to roughly half and thus on average we will need to try two polynomials $f(x)$ to find an irreducible one.

The implemented algorithm factored both keys n_{1a} and n_{1aa} , and found the factors as written in table 1.

Method	Modulus	Bound	Factor found
$p - 1$	n_{1a}	10^6	343829424673861057649675028924526719480359037 280956295040420246080868016693179912244171458 182429310465447847300832055477047542641820120 27455184284281277843
$p + 1$	n_{1aa}	10^7	526645390122669789462387195963296956483700357 011882308972655200768539072611077583239782963 572216338498096466345412053818913899704980576 764377382022974939892713651725406839879202190 596766705196104829870800000296287022812770342 507880227938889494768300380301333604981470837 2675024455696225019876088455507547597493
$p^2 + p + 1$	n_{1b}	10^6	497314523294662602519823028475446074909639171

Table 1: Factors found by Pollard's methods

The speed of the algorithm can be greatly increased by noting that $B! = \prod_{p \leq B} p^{\lfloor \log_p(B) \rfloor}$. This means that we only need to iterate over all primes $p \leq B$ and raise the current element to $p^{\lfloor \log_p(B) \rfloor}$ at each step.

According to the prime number theorem, the number of primes up to B is approximately $\frac{B}{\ln(B)}$, compared to B iterations in the original approach. This gives a speedup factor of $\approx \ln(B)$, which is ≈ 16 for $B = 10^7$. This method was also used in the submitted code.

1.2 Pollard's $p^2 + p + 1$ method

For Pollard's $p + 1$ method, we found that $(p - 1)(p + 1) = p^2 - 1$, resulting in a quadratic field. Similarly, we find that $(p - 1)(p^2 + p + 1) = p^3 - 1$ and thus we're now working with cubic fields, defined by the quotient group $G \cong \frac{\mathbb{F}_p^\times}{\mathbb{F}_p^\times}$ which has order $\frac{(p^3 - 1)}{p - 1} = p^2 + p + 1$. As before, we need an irreducible polynomial $f(x)$, however, now it is cubic; e.g. $f(x) = x^3 + ax + b$. One in three cubic roots is irreducible [1].

The result of the factorization of the key $n1b$ is also in table 1.

$p^2 + p + 1$ has the advantage over $p + 1$ that it is larger ($\approx p^2$ vs. $\approx p$) and thus more likely to be smooth. So, for a randomly chosen prime p as factor of N , the method is more likely to succeed. The trade-off however is that a larger smoothness bound B is required to factor a number of certain size, which makes each iteration slower. Compared to ECM, both $p + 1$ and $p^2 + p + 1$ are limited to their respective values being smooth, compared to ECM which allows many different elliptic curves over $\mathbb{F}_p$, each with a different order. Increasing the degree, d , of the extension field requires the bound to increase by $\propto p^d$ but increased the chances of it being smooth. The benefits are limited because the required bound scales exponentially fast.

2 Padding schemes

The problem is that we wish to decipher $c \equiv m^e \bmod N$, and we know that e is small and we know the repetitive structure of m .

The implemented attack makes use of Coppersmith's method of finding the small roots of a polynomial. We know $PAD(k) = k || H(k) || k || H(k) || \dots || 111 \dots 111$ is only unknown in k , and that the encryption was done with a relatively small exponent $e = 3$. The only problem is that we don't know $H(k)$, which causes the padding to not be linear in k . This is no problem however, as we can take $v = k || H(k)$ and point out that v gets repeated $t = \lfloor \log_2(N)/256 \rfloor - 1$ times. The asymptotic bound for Coppersmith's method to work is $k < N^{1/e}$, for a 1024-bit modulus and $e = 3$, this results in a maximum recoverable message of length ≈ 341 bits. Since our key k together with its hash $H(k)$ is only 256 bits, this method is able to retrieve v and find k from this. The recovered key for $c2a$ is 260475610619077747997499485920008829211.

However, for $e = 5$ or $e = 11$, we cannot recover the key via this method. This is because the maximum recoverable length would be $1024/5 \approx 205$ and $1024/11 \approx 93$ respectively, which is shorter than our 256-bit v . It is noteworthy however that for $e = 5$ the recoverable length is longer than the key k we're actually trying to find, suggesting there may be some other way to use Coppersmith in order to recover the key. I however did not find this method, but imagine it requires using a polynomial constructed purely in terms of k .

Trying to make this non-deterministic by using a random value r instead of $H(k)$ will not help the security against the Coppersmith attack, as the attack does not make use of this hash regardless. Also, since the AES-key k will only be encrypted once by RSA, realistically, you won't come across the same ciphertext twice.

To improve this padding scheme, one should use more independent randomness across the whole padded message and break the repetition. The current padding scheme has only 256 bits of actual unknowns, which is small enough to be decrypted if a small exponent is used during encryption. If the padded text was e.g. 896 random bits and a 128-bit key, you could not use Coppersmith because the unknown is too large. [3]

3 Attack using quantum computers

The method described in Shor's article [2], where the result is discarded if the order r is not even, is wasteful. We show that given the order r of any element $a \in (\mathbb{Z}/N\mathbb{Z})^\times$, one can almost always recover the factorisation of N classically, regardless of whether r is even or odd.

By Lagrange's theorem, the order of any element in a group divides the group order. For a prime p dividing N , the group $(\mathbb{Z}/p\mathbb{Z})^\times$ has order $p-1$, so $\text{ord}_p(a) \mid (p-1)$. Similarly, $\text{ord}_q(a) \mid (q-1)$ for another prime factor q . The order of a modulo N is then $r = \text{lcm}(\text{ord}_p(a), \text{ord}_q(a))$, so in particular both $\text{ord}_p(a)$ and $\text{ord}_q(a)$ divide r .

The key observation is the following: if we can find an exponent e such that $\text{ord}_p(a) \mid e$ but $\text{ord}_q(a) \nmid e$, then $a^e \equiv 1 \pmod{p}$ but $a^e \not\equiv 1 \pmod{q}$, and therefore $1 < \gcd(a^e - 1, N) < N$ yields a non-trivial factor of N .

To find such an exponent, we start with $e = r$ (for which $a^e \equiv 1 \pmod{N}$) and systematically divide e by small primes ℓ . For each trial, we compute $g = \gcd(a^{e/\ell} - 1, N)$:

- If $1 < g < N$: we have found a non-trivial factor.
- If $g = N$: both orders still divide e/ℓ , so we replace e with e/ℓ and continue.
- If $g = 1$: dividing by ℓ destroyed the divisibility for all prime factors, so we move on to the next prime ℓ .

This works because $\text{ord}_p(a)$ and $\text{ord}_q(a)$ are generically different, so their prime factorisations will differ at some prime ℓ . At that point, dividing out ℓ will kill the divisibility for one factor of N but not the other.

The algorithm fails only when $\text{ord}_p(a) = \text{ord}_q(a)$ for all prime factors p of N , in which case no divisor of r can separate the factors. For a randomly chosen a , this occurs with negligible probability. When N has more than two prime factors, the same procedure is applied recursively to any composite factors found.

The resulting prime factorizations of the provided moduli are given in table 2. Although in theory the algorithm should work for any modulus, I have found that it takes very long to factor some (e.g. n_{3c} and n_{3d}).

Modulus	Factors
n_{3a}	(9860061170105707872324609687636394302338377681968717896645702599267356386464395736133787923027648244488129957264614575335914118680790381294042095456220761), (16219713666811825216880660196769518240719404774698650168925497830948941534933065844136295950553046136629685910973417819575225366642642655851385102382609141)
n_{3b}	(16301269261912676521092286863017811358695576078393559333007274865269362816466634231492789714350288750109331189561246509133276151125195881386326935167422437), (21528184269826598990271285307463507476524429426198897202841849683980585300943081595414831671675540572224596901940160205923379938264166169546053906691829687), (23390946821276445154821831004849720548838569753544934037279943353356600490015885001237840792849096195083107744858259649682892361948170384317217727144579591), (23607715957043363159568002864656809707664339181274142896378999942725382627074301909621717105537292103260226048176359097128073137598165893822914611869261639)
n_{3c}	Wasn't able to factor
n_{3d}	Wasn't able to factor

Table 2: Factors by using Lagrange's theorem

4 A polynomial encryption scheme

The implemented method converts the problem into a Closest Vector Problem (CVP). The lattice is constructed with basis $f(x)$ and $g(x)$. We know the message $m(x)$ has small coefficients $\in \{0, 1\}$ and thus the ciphertext $c(x) = a(x)f(x) + b(x)g(x) + m(x)$ can be seen as a lattice point ($Lp = a(x)f(x) + b(x)g(x)$) and a small offset ($m(x)$). The original message m can then be found by subtracting the coordinate of the closest lattice point Lp from the ciphertext $c(x)$.

In the current implementation, there are bounds on the values of B and n . The coefficient bound of $f(x)$ and $g(x)$, B , has a lower bound in that it must be larger than one. This is because The CVP-method described above relies on $c(x)$ having a distinguishable closest lattice point. If $B = 1$, the "error" $m(x)$ becomes comparable to the lattice spacing, and $c(x)$ may be closer to another lattice point, leading to incorrect decryption. The bound U on the coefficients of $a(x)$ and $b(x)$ determines how far away from the origin the lattice point lies, which has no effect on the closest vector problem.

The bound on n comes from a computation limit. The current implementation uses the `ClosestVectors`-function in MAGMA, which calculates the exact solution to the closest vector problem. This scales exponentially in the lattice dimension $2n$, making it infeasible for $n > 128$. To improve this, one could use Babai's nearest plane algorithm on an LLL-reduced basis, which does run in polynomial time. This reduced basis could be augmented to the key generation, such that the only operation that needs to happen at decryption time is Babai's near plane algorithm, which is fast. This is an approximation, but works as long as B isn't too small and n isn't too large.

The attack can be performed by trying to reconstruct the lattice based on lattice points found by calculating $h_i(x) = c_i(x) - m_i(x) = a_i(x)f(x) + b_i(x)g(x)$. Since these are all lattice points spanned by the secret keys $f(x)$, $g(x)$ and their shifts $\{x^j f(x), x^j g(x)\}_{j=0}^{n-1}$, we know that the shortest vectors will be the keys, so we can perform any shortest vector algorithm such as LLL.

However, since the h_i have degree $2n - 1$, directly applying LLL yields short vectors that are combinations of f , g and their shifts, which are polynomials of degree $2n - 1$, not f and g themselves. To resolve this, the attack proceeds in two steps. In the first step, we find integer combinations of the h_i whose coefficients above degree n vanish. This is achieved by constructing a lattice with a large penalty on the high-degree coefficients and applying LLL; vectors where the penalty columns are zero correspond to combinations of degree $\leq n$. Each such polynomial is of the form $\alpha f + \beta g$ for some integers α, β since for the product $a(x)f(x)$ to have degree $\leq n$ with $\deg(f) = n$, the polynomial $a(x)$ must reduce to a constant. In the second step, we collect these degree- n polynomials and build a new lattice from their coefficient vectors in $\mathbb{Z}^{n+1}$. Since they are all integer linear combinations of f and g , this is a rank-2 lattice in which f and g are the shortest vectors. LLL then recovers them.

For this to work, we need at minimum $2n$ linearly independent points spread across the lattice, thus we need at least $2n$ ciphertext-plaintext pairs. In reality, we will need a bit more to account for the possible linear dependence of the random points.

The attack was tested for $B = 5$, $U = 100$, and $n = 16, 24, 32, 48, 64$ and succeeded in all cases. For this, I always used $\#pairs = 2.5n$. It is worthy to note that even though all attacks succeeded, the found keys f and g aren't per se the same ones as the original keys that were used. This is because all variations on the keypair (f, g) are also valid, e.g. $(-f, g), (f, -g), (g, f), \dots$. These are still able to decrypt the message.

This attack has its limitations. Mainly that it needs $> 2n$ ciphertext-plaintext pairs, as mentioned before. It also needs to be aware of the n , B , and U that were used during encryption. Although with LLL and Babai's nearest plane algorithm it would become polynomial in time ($\mathcal{O}(n^4 \log(UBn))$), it would still take very long for large n .

A ciphertext-only attack on the above cryptosystem would be significantly harder. The problem can then be seen as a Learning With Errors (LWE)-problem, in which $m(x)$ acts as noise with small coefficients $\{0, 1\}$ and trying to reconstruct the lattice from this. This problem is considered infeasible to solve in general.

References

- [1] Rudolf Lidl and Harald Niederreiter. *Finite Fields*. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 1997.
- [2] Peter W. Shor. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring”. In: IEEE, 1994. DOI: 10.1109/SFCS.1994.365700.
- [3] Yutong Zhong. “An Overview of RSA and OAEP Padding”. In: *Highlights in Science, Engineering and Technology* 1 (2022). DOI: 10.54097/hset.v1i.431.